实 验 报 告

课程名称	Python 程序设计	实验编号	实验一
实验名称	Python 安装与 Turtle 编程	实验学时	2 学时
姓　　名	安家乐	学　　号	3230212006
班　　级	工程 232	实验日期	2026 年 9 月 8 日

一、实验目的

1. 掌握 Python 的安装与环境配置方法，能独立搭建开发环境并运行程序。
2. 掌握顺序结构与循环结构的程序设计方法。
3. 掌握 Turtle（海龟）绘图模块的使用，能绘制并填充基本几何图形与组合图形。

二、实验环境

操作系统	Windows 11 64 位
Python 版本	Python 3.14.6
开发工具	IDLE / Visual Studio Code
使用模块	turtle（Python 标准库，无需额外安装）

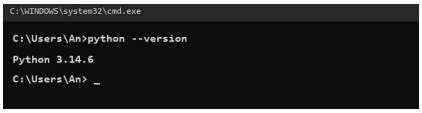
三、实验内容与步骤

3.1 Python 的安装与配置

（1）访问 python.org/downloads，下载 Windows x86-64 executable installer 安装包，安装时勾选 Add python.exe to PATH。

（2）安装后在命令行输入 python --version，返回 Python 3.14.6，说明安装成功。

（3）在项目目录用 python -m venv venv 创建虚拟环境以隔离依赖；turtle 属标准库，须用完整版解释器运行（见第四部分）。



3.2 绘制圆、等边三角形、正方形（test1.py）

（1）程序分析

本程序为顺序结构，三个图形重复同一动作序列：抬笔移动 → 落笔 → begin_fill → color → circle → end_fill。其中 circle(r) 画整圆，circle(r, steps=n) 用 n 段等长弦逼近圆，故 steps=3 得正三角形、steps=4 得正方形；填充由 begin_fill()/end_fill() 配对完成。

（2）源代码

```
import turtle
turtle.pensize(3)
turtle.penup()
turtle.goto(-100, -50)
turtle.pendown()
turtle.begin_fill()
turtle.color("red")
turtle.circle(40)
turtle.end_fill()
turtle.penup()
turtle.goto(0, -50)
turtle.pendown()
turtle.begin_fill()
turtle.color("blue")
turtle.circle(40, steps = 3)
turtle.end_fill()
```

```
turtle.penup()
turtle.goto(100, -50)
turtle.pendown()
turtle.begin_fill()
turtle.color("green")
turtle.circle(40, steps = 4)
turtle.end_fill()
turtle.hideturtle()
turtle.done()
```

（3）程序流程图与运行结果

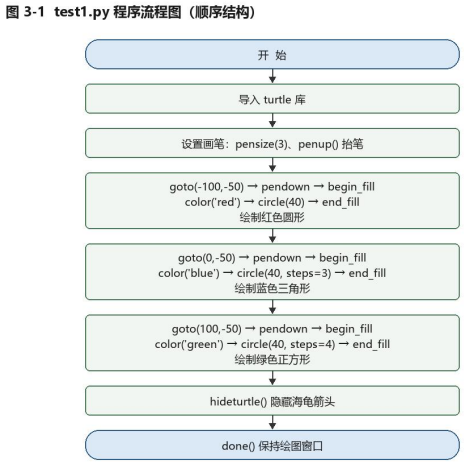


图 3-1 test1.py 流程图（顺序结构）



图 3-2 test1.py 运行结果（红圆 / 蓝三角 / 绿正方）

3.3 绘制正三、四、五、六、八边形（test2.py）

（1）程序分析

本程序用列表 SHAPES 与 for 循环逐个绘制 5 个正多边形。关键要求是底边平行于 x 轴：circle(R, steps=n) 的第 1 条弦方向为 heading + 180/n，令其等于 0° 得 heading = −180/n，即 setheading(-180/n)。指导书提示的 60° 只是 n=3 时 heading=180/n 的特例。布局上用 BASE_Y 对齐下沿、GAP（>2R）控制间距。

（2）源代码

```
import turtle
R = 40
SIDE_START_X = -220
GAP = 110
BASE_Y = -50
ROT = 0
turtle.pensize(3)
turtle.speed(3)
SHAPES = [
    (3, "red"),
    (4, "blue"),
    (5, "green"),
    (6, "orange"),
    (8, "purple"),
]
```

```
x = SIDE_START_X
for n, color in SHAPES:
    turtle.penup()
    turtle.goto(x, BASE_Y)
    turtle.pendown()
    turtle.begin_fill()
    turtle.color(color)
    turtle.setheading(-180 / n + ROT)
    turtle.circle(R, steps = n)
    turtle.end_fill()
    x = x + GAP
turtle.hideturtle()
turtle.done()
```

（3）程序流程图与运行结果

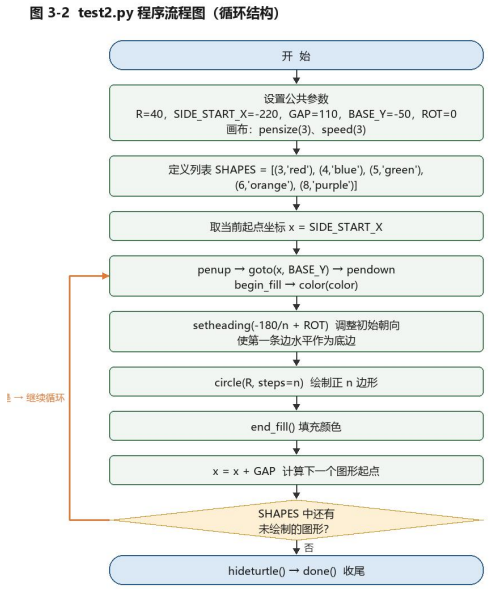


图 3-3 test2.py 流程图（循环结构）



图 3-4 test2.py 运行结果（五个正多边形底边全部水平、下沿对齐）

3.4 复杂图形：小猪佩奇（test3.py，选做）

（1）程序分析

采用“分解 + 图层顺序”法：将小猪分解为尾巴、前腿、身体、耳朵、头、猪鼻、鼻孔、眼睛共 8 组图元，按“先下后上”的顺序绘制，后画的覆盖先画的，自然遮住接缝。圆用 circle(r)，卷曲尾巴用 circle(r, extent)，鼻孔/眼睛用 dot(d, color)，矩形用 forward+right 折线。

（2）源代码

```
import turtle
turtle.pensize(3)
```

```
turtle.speed(3)
turtle.penup()
turtle.goto(75, -100)
turtle.pendown()
turtle.pencolor("hotpink")
turtle.pensize(6)
turtle.setheading(150)
turtle.circle(18, 300)
turtle.penup()
turtle.pensize(3)
turtle.pencolor("black")
for x in (-65, 31):
    turtle.goto(x, -160)
    turtle.setheading(0)
    turtle.pendown()
    turtle.begin_fill()
    turtle.color("pink")
    for i in range(2):
        turtle.forward(34)
        turtle.right(90)
        turtle.forward(55)
        turtle.right(90)
    turtle.end_fill()
    turtle.penup()
turtle.goto(0, -165)
turtle.setheading(0)
turtle.pendown()
turtle.begin_fill()
turtle.color("pink")
turtle.circle(80)
turtle.end_fill()
turtle.penup()
for x1, y1, x2, y2, x3, y3 in [(-60, 120, -20, 140, -75, 178),
                               ( 60, 120,  20, 140,  75, 178)]:
    turtle.penup()
    turtle.goto(x1, y1)
    turtle.pendown()
    turtle.begin_fill()
    turtle.color("hotpink")
    turtle.goto(x2, y2)
    turtle.goto(x3, y3)
    turtle.goto(x1, y1)
    turtle.end_fill()
    turtle.penup()
turtle.goto(0, -30)
turtle.setheading(0)
turtle.pendown()
turtle.begin_fill()
turtle.color("pink")
turtle.circle(90)
turtle.end_fill()
turtle.penup()
turtle.goto(0, -9)
turtle.setheading(0)
turtle.pendown()
turtle.begin_fill()
turtle.color("hotpink")
turtle.circle(34)
turtle.end_fill()
turtle.penup()
for x in (-13, 13):
    turtle.goto(x, 30)
    turtle.dot(12, "black")
for x in (-38, 38):
    turtle.goto(x, 85)
    turtle.dot(20, "black")
turtle.hideturtle()
turtle.done()
```

（3）程序流程图与运行结果

图 3-3 test3.py 程序流程图（小猪佩奇分层绘制）

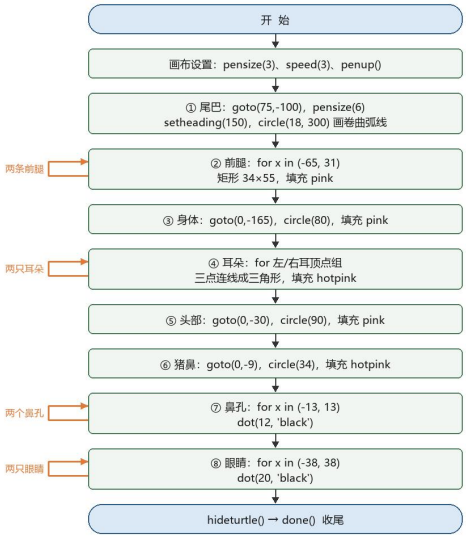


图 3-5 test3.py 流程图（分层绘制）



图 3-6 test3.py 运行结果（粉色卡通小猪）

四、实验中遇到的问题及解决方法

问题 1：venv 中 import turtle 报 ModuleNotFoundError

【现象】用虚拟环境解释器运行 test1.py 报 ModuleNotFoundError: No module named 'turtle'，改用 pip install turtle 也提示 No matching distribution。

【分析】turtle 是标准库，不需要也不能用 pip 单独安装；报错说明所用解释器不完整（托管/精简版未随附 tkinter 与 turtle）。

【解决】改用完整安装的系统 Python 3.14.6（C:\Users\An\AppData\Local\Programs\Python\Python314\python.exe）直接运行，三个程序均正常出图。

问题 2：多边形底边不水平（本实验关键错误）

【现象】test2.py 初版设 ROT = 60，运行后只有正六边形正常，正三角形尖端朝下（▽）、正方形呈菱形、正五/八边形均歪斜，不满足“底边与 x 轴平行”。

【分析】写校验脚本替换 turtle.done() 读取画布折线顶点，逐边算 Δy 定量判断水平边，结果如下表：

图形	ROT = 60（修改前）	ROT = 0（修改后）
正三角形	水平边在上方（▽）✗	第 1 条边水平且为底边 ✓
正方形	无水平边，呈菱形 ✗	第 1、3 条边水平 ✓
正五边形	无水平边 ✗	第 1 条边水平且为底边 ✓
正六边形	上下两边水平（巧合）✓	第 1、4 条边水平 ✓
正八边形	无水平边 ✗	第 1、5 条边水平 ✓

【原因】ROT = 60 是把“三角形朝向设为 60°”错误推广到所有边数；60° 只对 n=3 成立（180/3=60）。

【解决】由 heading + 180/n = 0 得 heading = −180/n，将 ROT 改为 0 后重新校验，五个图形底边全部水平、下沿对齐。

问题 3：图层顺序与填充配对

【现象】小猪腿、尾根露在身体外像“补丁”；个别图形颜色串色。

【分析】turtle 是“后画覆盖先画”，绘制顺序决定图层；begin_fill()/end_fill() 必须严格配对，否则后续填充区域被合并。

【解决】按“尾→腿→身→耳→头→鼻→鼻孔→眼”顺序绘制，并把腿、尾起点埋进身体；每个图形固定写成 begin_fill → color → circle → end_fill 四步，移动时 penup/pendown。

五、实验结果与分析

三个程序均在 Python 3.14.6 + turtle 下正常运行，结果汇总如下：

程序	结构类型	关键函数	实验结果
test1.py	顺序结构	circle(r)、circle(r, steps=n)、begin_fill/end_fill	红圆、蓝三角、绿正方，填充正常
test2.py	循环 + 列表	setheading(-180/n)、circle(R, steps=n)	五个正多边形，底边全部水平、下沿对齐
test3.py	循环 + 分层	circle(r, extent)、dot()、折线	完整小猪形象，图层无穿帮

结果分析：

（1）circle(R, steps=n) 本质是用 n 段等长弦逼近圆，n 越大越接近光滑圆弧，故“画多边形”与“画圆”共用同一函数。

（2）图形朝向完全由起笔朝向决定（第 1 条弦方向 = heading + 180/n），heading 是必须主动管理的状态量；复杂图形只需基本图元分解 + 图层顺序即可实现。

六、实验总结与心得

通过本次实验，我完成了 Python 环境的安装配置，编写运行了三个 turtle 程序，掌握了顺序结构与循环结构，理解了位置、朝向、画笔与填充等状态量的作用。最大收获是学会“先定量验证、再改代码”：排查底边不水平时，通过读取画布真实顶点坐标计算每条边 Δy 定位到错误根源，而非反复试角度；同时认识到指导书的提示角度往往是特例，应先推导通用公式（heading = −180/n）再代入数值。图层顺序、填充配对与参数化布局（BASE_Y/GAP/SIDE_START_X）等经验对今后的图形可视化工作同样通用。